

AVALIAÇÃO SOMATIVA

Programação Backend — JavaScript / Node.js

Caderno do Aluno • Versão Hands-on + Conceitual

Aluno(a)	
Turma	
Data	
Nota	_____ / 100

Formato da avaliação

Esta avaliação é composta por 10 exercícios práticos de programação, com dificuldade progressiva, e 5 questões conceituais.

Valor total: 100 pontos (80 pontos práticos + 20 pontos conceituais).

Todo código deverá ser executado e testado no Node.js.

Ao final, todos os arquivos deverão ser enviados para um repositório no GitHub e o link deverá ser informado na última página.

Conteúdos avaliados

- ☐ Variáveis, operadores e saída com `console.log()`.
- ☐ Entrada de dados no terminal com `readline-sync`.
- ☐ Estruturas condicionais: `if`, `else if` e `else`.
- ☐ Estruturas de repetição: `for` e/ou `while`.
- ☐ Arrays: criação, `push()`, acesso por índice e `length`.
- ☐ Objetos JavaScript e acesso às propriedades.
- ☐ Funções com parâmetros e retorno.
- ☐ Integração entre arrays, objetos, laços e funções.
- ☐ Modularização simples com `require()` e `module.exports`.
- ☐ Compreensão conceitual dos recursos utilizados nos programas.

Importante

A avaliação está limitada aos conteúdos descritos neste caderno. Resolva os exercícios usando as estruturas e recursos trabalhados em aula.

Pontuação

Exercício	Tema principal	Valor
1	Variáveis e cálculo	4 pts
2	Entrada de dados	4 pts
3	Condicional simples	6 pts
4	Condicional múltipla	6 pts
5	Laço de repetição	8 pts
6	Repetição e acumuladores	8 pts
7	Arrays	10 pts
8	Arrays e objetos	10 pts
9	Funções	12 pts
10	Modularização	12 pts
	SUBTOTAL PRÁTICA	80 pts
5 questões	Questões conceituais	20 pts
	TOTAL DA AVALIAÇÃO	100 pts

Objetivo: Revisar variáveis, operações matemáticas e saída de dados.

Uma linha de produção fabrica uma determinada quantidade de peças por hora. Crie um programa que calcule quantas peças serão produzidas em um turno.

O programa deve:

- ☐ Criar uma variável com a quantidade de peças produzidas por hora.
- ☐ Criar uma variável com a quantidade de horas do turno.
- ☐ Calcular a produção total do turno.
- ☐ Exibir uma frase informando a produção por hora, as horas e o total produzido.

Arquivo para entrega

Salve este exercício como: `ex01_producao.js`

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Use, pelo menos uma vez, os valores 120 peças/hora e 8 horas. O resultado deverá ser 960 peças.

Como os pontos serão distribuídos

Critério	Pontos
Variáveis criadas corretamente	1
Cálculo correto	1
Saída clara no console	2

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	____ / 4

Objetivo: Utilizar entrada de dados no terminal e operações com valores numéricos.

Crie um programa para calcular o custo de uma compra de matéria-prima.

O programa deve:

- ☐ Importar a biblioteca readline-sync.
- ☐ Solicitar ao usuário o nome do material.
- ☐ Solicitar a quantidade comprada.
- ☐ Solicitar o preço unitário.
- ☐ Calcular o valor total da compra.
- ☐ Exibir um pequeno resumo da compra.

Arquivo para entrega

Salve este exercício como: ex02_compra.js

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Exemplo: 10 unidades a R\$ 12,50 devem resultar em R\$ 125,00.

Como os pontos serão distribuídos

Critério	Pontos
Entradas funcionando	1
Cálculo correto	2
Resumo exibido corretamente	1

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	_____ / 4

Objetivo: Aplicar uma estrutura condicional simples.

Uma peça será aprovada no controle de qualidade quando seu peso estiver entre 95 g e 105 g, inclusive. Crie um programa que leia o peso e informe o resultado da inspeção.

O programa deve:

- ☐ Solicitar o peso da peça.
- ☐ Usar if/else para decidir se a peça está dentro do padrão.
- ☐ Exibir 'PEÇA APROVADA' quando estiver entre 95 e 105 g.
- ☐ Exibir 'PEÇA REPROVADA' nos demais casos.
- ☐ Exibir também o peso informado.

Arquivo para entrega

Salve este exercício como: ex03_qualidade.js

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Teste com 100 g e com 110 g.

Como os pontos serão distribuídos

Critério	Pontos
Entrada de dados	1
Condição correta	3
Mensagens corretas	2

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	_____ / 6

Objetivo: Trabalhar com if, else if e else em uma regra de negócio.

Crie um programa que leia a temperatura de uma máquina e classifique sua situação.

O programa deve:

- ☐ Até 60 °C: situação NORMAL.
- ☐ De 61 °C até 80 °C: situação ATENÇÃO.
- ☐ Acima de 80 °C: situação CRÍTICA.
- ☐ Solicitar a temperatura pelo terminal.
- ☐ Exibir a temperatura e a classificação.

Arquivo para entrega

Salve este exercício como: ex04_temperatura.js

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Teste com 50 °C, 70 °C e 90 °C.

Como os pontos serão distribuídos

Critério	Pontos
Entrada de dados	1
Faixas de decisão corretas	3
Saída clara	2

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	____ / 6

Objetivo: Utilizar laço de repetição para gerar uma sequência de resultados.

Uma máquina produz uma quantidade fixa de peças a cada ciclo. Crie um programa que mostre a produção acumulada do ciclo 1 até o ciclo 10.

O programa deve:

- ☐ Solicitar quantas peças a máquina produz por ciclo.
- ☐ Utilizar um laço for para percorrer os ciclos de 1 até 10.
- ☐ Em cada ciclo, exibir o número do ciclo e a produção acumulada.
- ☐ Não escrever manualmente as dez linhas.

Arquivo para entrega

Salve este exercício como: ex05_ciclos.js

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Se forem 15 peças por ciclo: ciclo 1 = 15; ciclo 5 = 75; ciclo 10 = 150.

Como os pontos serão distribuídos

Critério	Pontos
Entrada de dados	1
Laço de repetição correto	3
Cálculo acumulado correto	2
Exibição organizada	2

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	_____ / 8

Objetivo: Combinar repetição, entrada de dados e acumulador.

Durante um teste de processo, são realizadas cinco medições. Crie um programa que leia as cinco medições e calcule a média.

O programa deve:

- ☐ Criar uma variável acumuladora iniciada em zero.
- ☐ Usar um laço para solicitar exatamente 5 valores.
- ☐ Somar cada valor ao acumulador.
- ☐ Ao final, calcular a média.
- ☐ Exibir a soma das medições e a média final.

Arquivo para entrega

Salve este exercício como: `ex06_media_medicoes.js`

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Teste com 10, 20, 30, 40 e 50. A soma será 150 e a média será 30.

Como os pontos serão distribuídos

Critério	Pontos
Laço executando 5 vezes	2
Acumulador funcionando	2
Cálculo da média	2
Saída final correta	2

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	_____ / 8

Objetivo: Cadastrar e percorrer dados armazenados em um array.

Crie um programa para cadastrar os nomes de cinco operadores de uma equipe e, ao final, listar todos os nomes numerados.

O programa deve:

- ☐ Criar um array vazio.
- ☐ Usar um laço para solicitar 5 nomes.
- ☐ Adicionar cada nome ao array usando push().
- ☐ Depois do cadastro, percorrer o array novamente.
- ☐ Exibir no formato: '1 - Nome', '2 - Nome' etc.
- ☐ Utilizar a propriedade length em pelo menos um dos laços.

Arquivo para entrega

Salve este exercício como: ex07_operadores.js

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Cadastre cinco nomes diferentes e confirme se todos aparecem na listagem final.

Como os pontos serão distribuídos

Critério	Pontos
Array e push()	2
Cadastro com laço	2
Percurso do array	2
Uso de length e saída numerada	4

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	____ / 10

Objetivo: Combinar objetos, arrays, repetição e condição em um único programa.

Crie um programa para cadastrar três componentes do estoque. Cada componente deverá possuir nome, quantidade e estoque mínimo. Ao final, o programa deverá apresentar todos os componentes e informar quais precisam de reposição.

O programa deve:

- ☐ Criar um array vazio para armazenar os componentes.
- ☐ Cadastrar 3 componentes usando um laço.
- ☐ Para cada componente, criar um objeto com nome, quantidade e estoqueMinimo.
- ☐ Adicionar cada objeto ao array com push().
- ☐ Percorrer o array com um laço.
- ☐ Se quantidade for menor que estoqueMinimo, exibir 'REPOR ESTOQUE'.
- ☐ Caso contrário, exibir 'ESTOQUE OK'.

Arquivo para entrega

Salve este exercício como: ex08_estoque.js

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Cadastre pelo menos um item abaixo e um item acima do estoque mínimo.

Como os pontos serão distribuídos

Critério	Pontos
Criação dos objetos	2
Armazenamento no array	2
Cadastro com repetição	2
Percurso e condição corretos	2
Relatório final	2

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	_____ / 10

Objetivo: Organizar regras do programa em funções com parâmetros e retorno.

Crie um programa que calcule a eficiência de uma produção. A eficiência é dada por: (produção real / produção prevista) × 100.

O programa deve:

- ☐ Criar uma função `calcularEficiencia(real, prevista)` que retorne o percentual.
- ☐ Criar uma função `classificarEficiencia(percentual)` que retorne uma classificação.
- ☐ Classificação: 90% ou mais = 'META ATINGIDA'; de 70% a 89,99% = 'ATENÇÃO'; abaixo de 70% = 'ABAIXO DA META'.
- ☐ Solicitar produção prevista e produção real pelo terminal.
- ☐ Chamar as duas funções.
- ☐ Exibir produção prevista, produção real, percentual e classificação.

Arquivo para entrega

Salve este exercício como: `ex09_eficiencia.js`

Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Prevista = 1000 e real = 950 → 95% e META ATINGIDA.
- Prevista = 1000 e real = 650 → 65% e ABAIXO DA META.

Como os pontos serão distribuídos

Critério	Pontos
Função <code>calcularEficiencia</code>	3
Função <code>classificarEficiencia</code>	3
Entrada e chamadas das funções	2
Relatório final correto	4

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	_____ / 12

Objetivo: Separar funções de negócio em um módulo e utilizá-las em um programa principal.

Uma oficina industrial precisa calcular o orçamento de manutenção de uma máquina. Desenvolva a solução utilizando dois arquivos JavaScript.

O programa deve:

- ☐ Criar uma pasta chamada ex10_manutencao.
- ☐ Criar o arquivo funcoesManutencao.js.
- ☐ Nesse arquivo, criar calcularMaoDeObra(horas), considerando R\$ 80,00 por hora.
- ☐ Criar calcularTotal(valorPecas, horas), somando peças e mão de obra.
- ☐ Criar verificarGarantia(meses), retornando 'EM GARANTIA' quando meses ≤ 6 e 'FORA DA GARANTIA' nos demais casos.
- ☐ Exportar as funções usando module.exports.
- ☐ Criar o arquivo app.js.
- ☐ No app.js, importar readline-sync e o módulo funcoesManutencao.js usando require().
- ☐ Solicitar nome da máquina, valor das peças, horas de serviço e meses desde a última manutenção.
- ☐ Exibir um relatório final com máquina, mão de obra, peças, total e situação da garantia.

Arquivo para entrega

Salve este exercício como: ex10_manutencao/app.js + ex10_manutencao/funcoesManutencao.js
Execute o arquivo pelo terminal com Node.js e confirme seu funcionamento.

Teste mínimo

- Peças = R\$ 200,00 e 2 horas → mão de obra = R\$ 160,00 e total = R\$ 360,00.
- Meses = 4 → EM GARANTIA; meses = 8 → FORA DA GARANTIA.

Como os pontos serão distribuídos

Critério	Pontos
Funções corretas no módulo	4
module.exports e require funcionando	2
Entradas e integração no app.js	3
Relatório e cálculos corretos	3

Resultado da execução conferido pelo aluno	☐ Sim ☐ Não
Pontuação obtida	_____ / 12

PARTE B — QUESTÕES CONCEITUAIS

Valor: 20 pontos — 5 questões de 4 pontos cada.

Responda com suas próprias palavras. Quando solicitado, use exemplos relacionados aos programas desenvolvidos durante a avaliação.

1. Entrada de dados e variáveis

Explique para que serve uma variável em JavaScript e por que, ao receber um número digitado pelo usuário com `readline-sync`, pode ser necessário convertê-lo para um valor numérico antes de realizar cálculos.

Resposta: Existe dois tipos de variável, `const` e `let`, `const` serve para deixar imutável a variável durante o programa e `let` deixa o valor mutável durante o programa, serve para armazenar diversos tipos de valores, ao receber um numero do usuário deve se converter para `int` ou `float` porque por padrão vem como `string` (texto), se deve fazer isso para realizar cálculos.

2. Estruturas condicionais

Qual é a finalidade de uma estrutura `if / else`? Explique como ela permite que um programa tome decisões e dê um exemplo simples de uma condição que poderia ser usada em um sistema industrial.

Resposta: `if` e `else` serve para dar lógica ao sistema “se isso for verdade faça aquilo, se for mentira faça aquele” o “se” é o `if` e o “se não” o `else`, em um sistema industrial o exemplo é a temperatura SE for maior de 100° ligar para bombeiro SE NÃO continuar o trabalho

3. Estruturas de repetição

Explique a vantagem de utilizar um laço de repetição, como `for` ou `while`, em vez de escrever várias vezes o mesmo bloco de código. Cite uma situação da prova em que um laço é apropriado.

Resposta: utilizar laços no programa diminui o tamanho do código e deixa escalável para futuras melhorias, ajuda a não ter que escrever varias vezes, por exemplo se tenho uma lista de 1000 item ao em vez de lista cada uma eu crio uma lista e simplifica o trabalho

4. Arrays e objetos

Explique, de forma simples, a diferença entre um array e um objeto em JavaScript. Em um cadastro de componentes de estoque, qual seria a função do array e qual seria a função de cada objeto armazenado nele?

Resposta: lista guarda informações já o objeto é mais complexo é como uma lista dentro de uma lista, em um ambiente de estoque criase uma lista principal dentro dele crio meus objetos e cada objeto tem seus próprios valores e dados a função do objeto é dividir de forma estratégica e organizado os valores/dados

5. Funções e modularização

Explique o que são parâmetros e retorno de uma função. Depois, explique para que servem `module.exports` e `require()` quando organizamos um programa em mais de um arquivo JavaScript.

Resposta: parametros sao função() tudo dentro do () é meu parâmetro é a informação esperada que precisa para função ser feita, `module.exports` serve para exportar para outros arquivos na mesma pagina minhas funções, e o `require()` é o que eu estou pedindo (requirindo), precisa para importa minhas funções, o que torna possível eu usar minhas funções em mais de um arquivo javascript

ENTREGA FINAL — GITHUB

Organização sugerida do repositório

```
avaliacao-pbe1/  
├── ex01_producao.js  
├── ex02_compra.js  
├── ex03_qualidade.js  
├── ex04_temperatura.js  
├── ex05_ciclos.js  
├── ex06_media_medicoes.js  
├── ex07_operadores.js  
├── ex08_estoque.js  
├── ex09_eficiencia.js  
└── ex10_manutencao/  
    ├── app.js  
    └── funcoesManutencao.js
```

Checklist de entrega

- ☐ Todos os 10 exercícios foram criados.
- ☐ Todos os programas foram executados pelo menos uma vez.
- ☐ Os arquivos estão organizados e com os nomes solicitados.
- ☐ O exercício 10 possui os dois arquivos e funciona com require/module.exports.
- ☐ O repositório foi enviado ao GitHub.
- ☐ O link abaixo abre corretamente.

LINK DO REPOSITÓRIO NO GITHUB

Data/hora do último envio: _____

TOTAL DA AVALIAÇÃO: 100 PONTOS

Parte prática	Parte conceitual	Nota final
_____ / 80	_____ / 20	_____ / 100